SWE316 (Software Design and Construction)

Homework # 1

Date of submission: 10/23/2025

Name: Ibrahim Alshayea

ID: 202176470

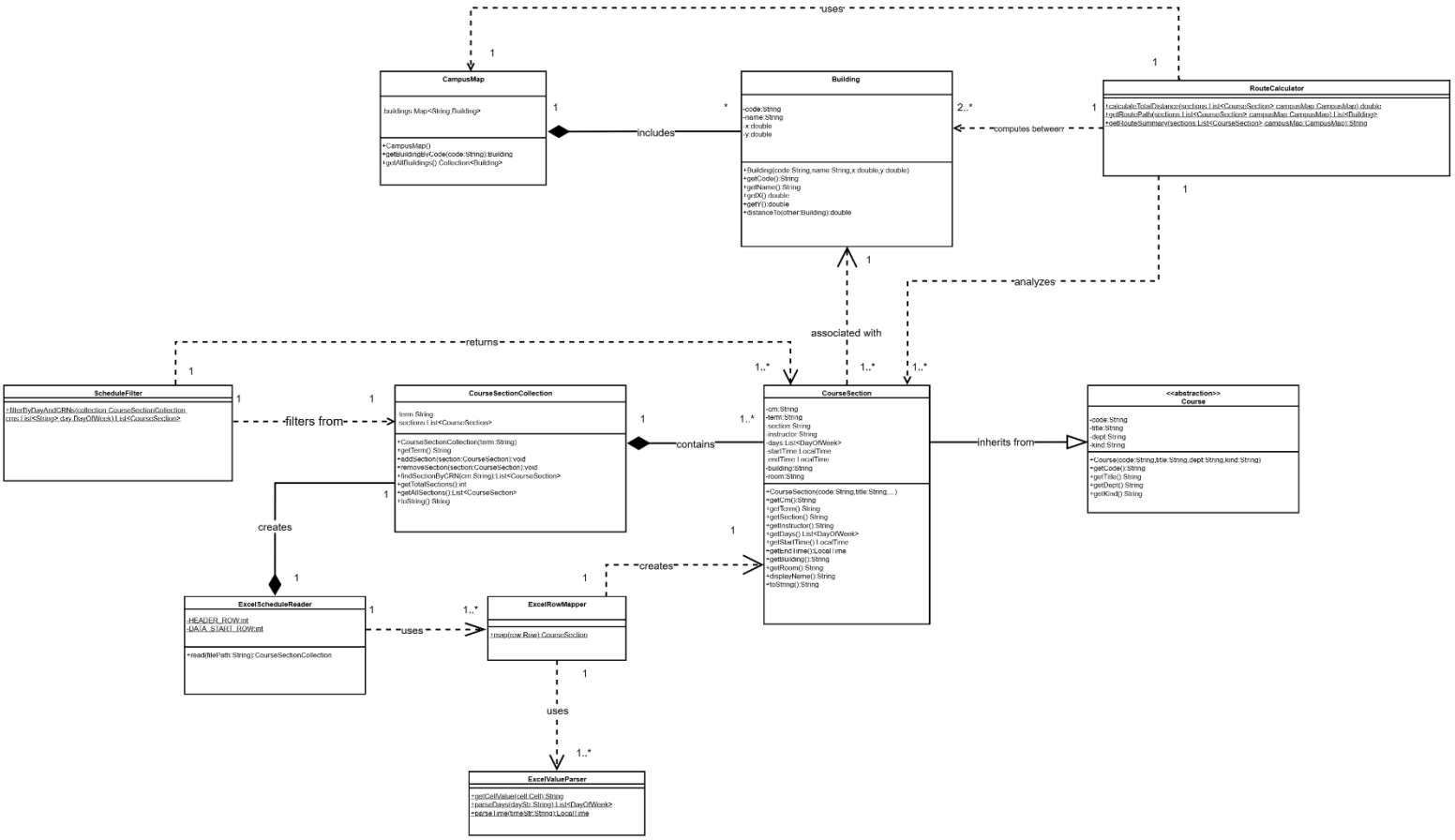
Task	Grade	Your Grade	Comments
Task # 1: Class Diagram	15		
Task # 2: Application	55		
Check list and penalties			
No Cover page with grade table	-5	☐	
File name (report)	-5	☐	
Not in PDF format	-5	☐	
Total	70		

1. UML Class Diagram (Task#1)

The class diagram is shown below.

It models the relationships between key components:

- **Course / CourseSection / CourseSectionCollection represent academic data.**
- **Building / CampusMap handle spatial representation.**
- **RouteCalculator / ScheduleFilter manage logic and filtering.**
- **ExcelValueParser / ExcelRowMapper / ExcelScheduleReader handle Excel data input.**



For High Resolution View, Click Here

Class Responsibility Summary

Class	Responsibility
Course (abstract)	Base abstraction for shared course attributes.
CourseSection	Represents a specific section (CRN, schedule, time, building).
CourseSectionCollection	Stores and manages all course sections for a term.
Building	Represents a building's code, name, and coordinates.
CampusMap	Loads and stores all buildings; supports building lookup.
RouteCalculator	Computes routes and total distance between class buildings.
ScheduleFilter	Filters sections by CRNs and day of week.
ExcelValueParser	Parses cell content, time, and day formats.
ExcelRowMapper	Maps Excel rows into CourseSection objects.
ExcelScheduleReader	Reads Excel file and constructs CourseSectionCollection.

2. Implementation Highlights (Task #2)

1. Course (abstract)

Purpose:

Defines shared course properties like code, title, department, and type.

Constructor:

```
public  Course(String code, String title, String dept, String kind){
    this.code = code;
    this.title = title;
    this.dept = dept;
    this.kind = kind;
}
```

2. CourseSection

Purpose:

Extends Course to represent a specific class section, with CRN, term, instructor, schedule, and location.

Constructor:

```
public CourseSection(String code, String title, String dept, String kind,
                     String crn, String term, String section, String instructor,
                     List<DayOfWeek> days, LocalTime startTime, LocalTime endTime,
                     String building, String room) {
    super(code, title, dept, kind);
    this.crn = crn;
    this.term = term;
    this.section = section;
    this.instructor = instructor;
    this.days = days;
    this.startTime = startTime;
    this.endTime = endTime;
    this.building = building;
    this.room = room;
}
```

3. CourseSectionCollection

Purpose:

Stores and manages CourseSection objects for one term. Supports adding, searching, and filtering.

Important Code:

Metod to find a specific section by inputting a CRN

```java
public List<CourseSection> findSectionsByCRNs(List<String> crns) {
    if (crns == null || crns.isEmpty()) return new ArrayList<>();
    return sections.stream()
            .filter(s -> crns.contains(s.getCrn()))
            .collect(Collectors.toList());
}
```

4. Building

Purpose:

Represents a physical building on campus with coordinates for distance calculations.

Important Code, the constructor and a method to calculate the distance to another building:

```java
public Building(String code, String name, double x, double y) {
    this.code = code;
    this.name = name;
    this.x = x;
    this.y = y;
}
1 usage  new *
public double distanceTo(Building other) {
    double dx = x - other.x;
    double dy = y - other.y;
    return Math.sqrt(dx * dx + dy * dy);
}
```

5. CampusMap

Purpose:

Manages a collection of Building objects and provides lookup by building code.

Constructor:

```java
public class CampusMap {

    39 usages
    private final Map<String, Building> buildings = new HashMap<>();

    2 usages   new *
    public CampusMap() {
        buildings.put("63", new Building( code: "63",  name: "Building 63",  x: 137,  y: 243));
        buildings.put("26", new Building( code: "26",  name: "Building 26",  x: 65,  y: 269));
        buildings.put("75", new Building( code: "75",  name: "Building 75",  x: 107,  y: 302));
        buildings.put("1",  new Building( code: "1",  name: "Building 1",  x: 88,  y: 337));
```

The code for some of the following classes is long so I can't show it here. If you want to take a look you can find the code in the folder.

6. RouteCalculator

Purpose:

Calculates total distance and route paths between classes based on buildings.

The class has two main methods-calculateTotalDistance() and getRouteSummary().

7. **ScheduleFilter**

Purpose:

Filters sections by CRNs and day of week.

Constructor and Method:

```java
public class ScheduleFilter {
    2 usages   new *
    public static List<CourseSection> filterByDayAndCRNs(
            CourseSectionCollection collection,
            List<String> crns,
            DayOfWeek day) {
        return collection.getAllSections().stream()
                .filter(s -> crns.contains(s.getCrn()) && s.getDays().contains(day))
                .collect(Collectors.toList());
    }
}
```

7. ExcelValueParser

Purpose:

Converts Excel cell data into usable Java formats like String, DayOfWeek, and LocalTime.

The class has three main methods:

getCellValue(), parseDays(), and parseTime() methods.

8. ExcelRowMapper

Purpose:

Maps a row in the Excel file to a CourseSection object.

Main method:

map(Row row) method.

9. ExcelScheduleReader

Purpose:

Reads an Excel file and constructs a CourseSectionCollection using the row mapper.

Main method :

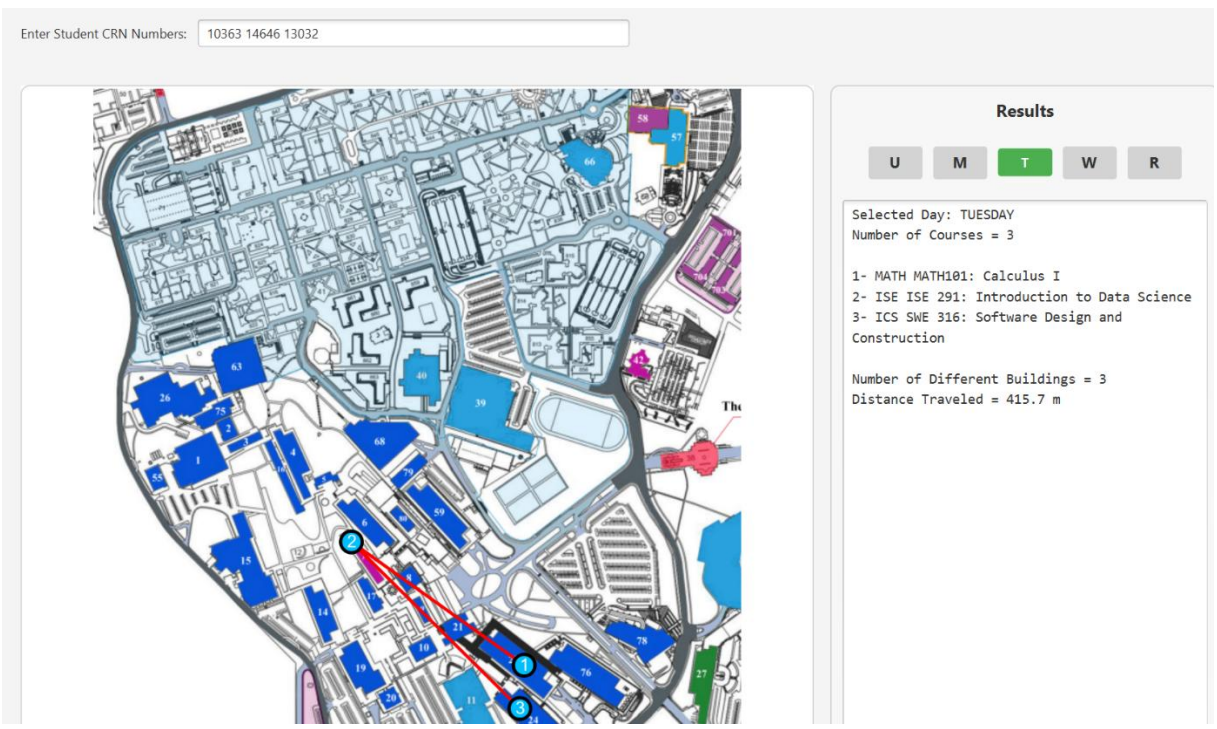
read(filePath) method.

3. User Interface using JavaFX

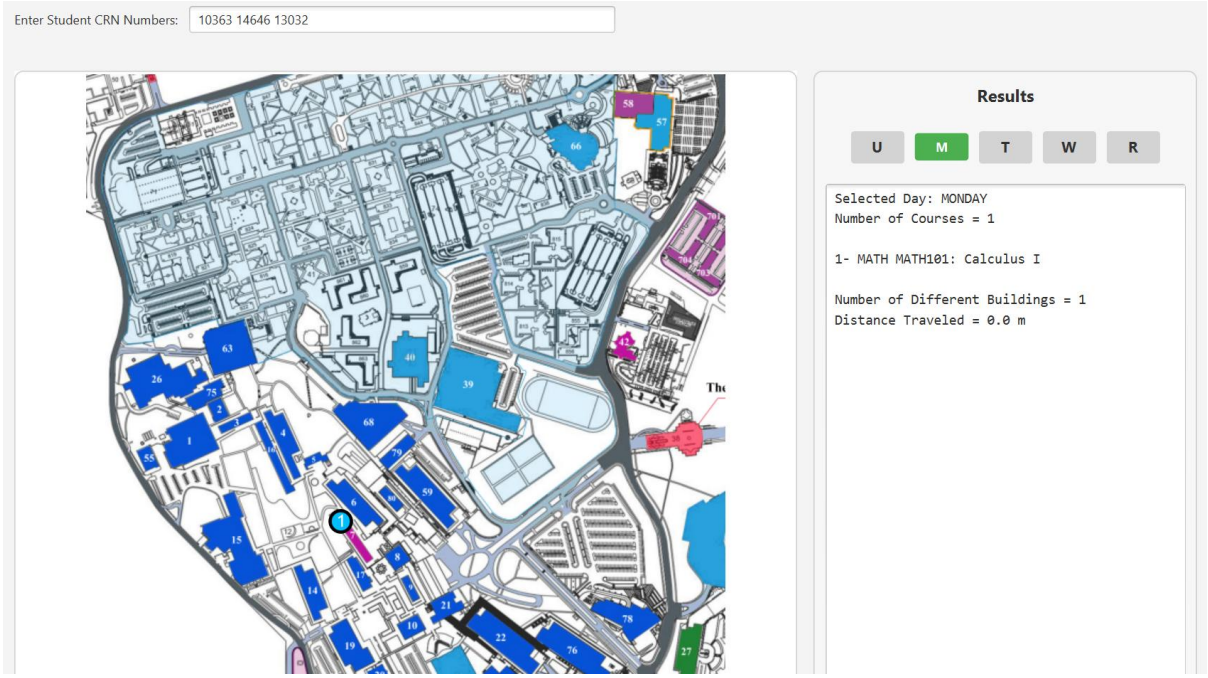
The user interface allows the user to input a list of CRNs, select a day of the week, and visualize the walking route between class buildings on the KFUPM campus map.

Examples:

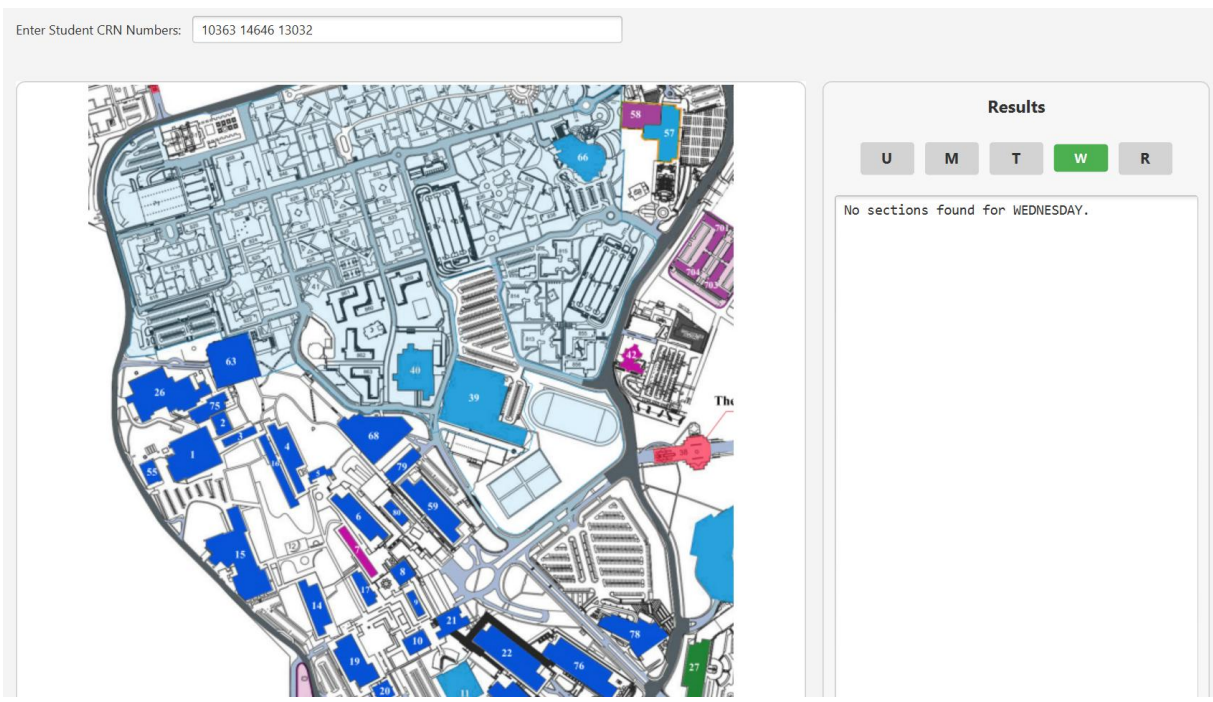
- Example1:



- Example2:



- Example3:



- Example4:

Enter Student CRN Numbers: 10363 14646 13032 14852 11717 10139

Results

U M T W R

```
Selected Day: TUESDAY
Number of Courses = 6

1- PMP MATH001: Preparatory Mathematics I
2- MATH MATH101: Calculus I
3- ISE ISE 291: Introduction to Data Science
4- ICS SWE 316: Software Design and
Construction
5- ACD SSC 201: Introduction to Smart &
Sustainable Cities
6- CHE CHE 306: Separation Processes

Number of Different Buildings = 6
Distance Traveled = 1080.1 m
```